

Habitus — техническое задание разработчику

Бэкенд, Go-шлюз · три задачи по порядку · выполняются через Claude Code · август 2026

Код пишет Claude Code. Работа разработчика — точно поставить задачу, проверить результат и не пустить в основную ветку мусор. Документ устроен так, чтобы обе части были механическими: каждая задача содержит готовый промт для копирования и список проверок, каждая из которых либо проходит, либо нет.

Главное правило: не верить на слово. Claude Code пишет уверенно даже когда ошибается. Задача считается сделанной не тогда, когда он написал «готово», а тогда, когда прошли все пункты приёмки. Если хотя бы один пункт не прошёл — задача не сделана, независимо от того, что написано в ответе.

Что предстоит

Три задачи в Go-шлюзе (папка `backend/`). Делаются **строго по порядку**: каждая следующая опирается на понимание, полученное в предыдущей. Фронтенд и Python-часть в этих задачах не участвуют вообще.

№	ЗАДАЧА	СУТЬ	ОЦЕНКА
1	Честная проверка здоровья сервиса	<code>/health</code> сейчас отвечает «всё хорошо», даже когда база лежит	1 день
2	Фильтры для объявлений на карте	Карта тянет всё подряд, включая объекты без фотографий	1–2 дня
3	Эндпоинт статистики данных	Состояние базы сейчас можно узнать только запросом руками	1–2 дня

Об архитектуре — минимум, который нужен

Три слоя, поднимаются одной командой `docker compose up`:

- **frontend** (порт 3000) — сайт. В этих задачах не трогаем.
- **backend** (порт 8080) — **наш Go-шлюз**. Публичный API `/api/v1`. Ходит в ML-сервис и в базу.
- **ml-service** (порт 8000) — Python, поиск и досье. Внутренний, наружу не смотрит. В этих задачах не трогаем.
- База — PostgreSQL на порту 5544.

Внутри `backend/` слои разделены строго, и это разделение надо соблюдать: `internal/repository/` — запросы к базе, `internal/service/` — логика, `internal/http/handlers/` — разбор запроса и ответ, `internal/http/router.go` — список маршрутов. Логiku в обработчик не класть, SQL в сервис не тащить.

Важное свойство таблицы объявлений: её владелец — Python-часть, Go в неё только читает. В файле `internal/repository/listing_repo.go` это написано первой строкой. Никаких `INSERT`, `UPDATE` и `DELETE` по таблице `listings` из Go быть не может ни в одной из трёх задач.

Общие правила — действуют во всех трёх задачах

ВЕТКА И КОММИТЫ

- Работаем прямо в `main`. Отдельные ветки не заводим.
- **Перед началом каждого рабочего дня:** `git pull`.
- Сообщение коммита — по-русски, с префиксом: `feat:` новая возможность, `fix:` исправление, `test:` тесты, `docs:` документация, `refactor:` перестройка без смены поведения.
- **Никаких подписей и трейлеров.** Ни `Co-Authored-By`, ни упоминаний Claude. Появилась такая строка — попросить убрать и переделать коммит.
- Одна задача — один коммит.

ЧТО НЕЛЬЗЯ ТРОГАТЬ НИ ПРИ КАКИХ ОБСТОЯТЕЛЬСТВАХ

- `backend/internal/cian/` — сборщик объявлений. Ходит на внешний сайт; неаккуратный эксперимент может лишить нас доступа к данным. Он лежит в той же папке `backend/`, поэтому предупреждение не лишнее.

- `habitus/` — вся Python-часть, включая схему базы `habitus/db/schema.sql`. Схему в этих задачах не меняем: всё, что нужно посчитать и отфильтровать, уже есть в существующих колонках.
- `frontend/` — не наша область.
- `.env` — файл с ключами. **Никогда не коммитить и не показывать в переписке.** Появился в `git status` — остановиться и написать Ярику.

ПРАВИЛО ПРОЕКТА, КОТОРОЕ НАРУШАЮТ ЧАЩЕ ВСЕГО

Не выдумывать факты. Если данных по объекту нет — поле не отдаётся вовсе. Подставлять ноль вместо отсутствующего значения запрещено: ноль означает «измерили и получили ноль», а не «не знаем». Это не стилистика, а основа продукта — мы обещаем пользователю, что всё показанное является реальными данными. Если Claude Code предложил «вернём 0, чтобы не было null» — это неверный ответ.

ТЕСТЫ

Каждая задача включает тесты — не «если успеем». Хорошая новость: тесты Go-шлюза **не требуют базы**, они работают на подставных объектах и запускаются за секунды:

```
cd backend && go test ./...
```

Коммит делается только когда всё зелёное. Если тест падает — чинится код, а не удаляется тест.

Нулевой день: поднять проект

Делается один раз. Пока не сделано — к задачам не приступать.

ЧТО ПОПРОСИТЬ У ЯРИКА ЗАРАНЕЕ

- Файл `.env` с ключами — его нет в репозитории.

ШАГИ

```
git clone <репозиторий> && cd Habitus
cp .env.example .env           # затем вписать ключи, которые дал Ярик
docker compose up              # первый запуск долгий: тянутся образы и модели

# наполнить базу объявлениями (файл уже лежит в репозитории), в отдельном окне:
uv run habitus offline --csv listings.csv --source cian
```

Проверка, что шлюз жив: `curl localhost:8080/health` отвечает `{"status":"ok"}`.

Если при открытии `localhost:3000` не отрисовалась карта — это нормально и вас не касается: для карты нужен отдельный ключ, который в бэкенд-задачах не нужен. Проверять свою работу вы будете через `curl`, а не через сайт.

КАК ОБРАЩАТЬСЯ К АРІ ИЗ ТЕРМИНАЛА

Почти все ручки закрыты авторизацией по куке. Один раз заводим пользователя и сохраняем куку в файл, дальше подставляем её в каждый запрос:

```
curl -s -X POST localhost:8080/api/v1/auth/register \
-H 'Content-Type: application/json' \
-d '{"email":"dev@local","password":"devdevdev","name":"dev"}' \
-c /tmp/habitus.cookie

# дальше во все запросы добавляем -b /tmp/habitus.cookie:
curl -s -b /tmp/habitus.cookie \
'localhost:8080/api/v1/geo/listings?bbox=37.5,55.70,37.70,55.80' \
| python3 -m json.tool | head -40
```

Если пользователь уже заведён, вместо `register` используем `login` с теми же почтой и паролем.

КАК РАЗГОВАРИВАТЬ С CLAUDE CODE

- Промт копируется **целиком**, включая разделы «Границы» и «Правила». Куски выкидывать нельзя — они там ровно для того, чтобы он не полез в чужие файлы.
- Первый ответ должен быть **планом, а не кодом**. Так написано в конце каждого промта. Прочитайте план: в нём должны быть только файлы из раздела «Границы». Появились другие — не соглашаться, ответить: «файлы вне списка не трогай».
- Две попытки починить одно и то же не помогли — **не крутить дальше**, писать Ярику с полным текстом ошибки.

Задача 1 — Честная проверка здоровья сервиса

ОБЛАСТЬ	ОЦЕНКА	ПОРЯДОК
Go, backend/	1 день	Первая

ЗАЧЕМ

Сейчас ручка `/health` состоит из одной строки и **всегда** отвечает `{"status":"ok"}` — даже если база недоступна и сайт полностью не работает. Это хуже, чем отсутствие проверки: мониторинг будет показывать зелёное, пока пользователи видят ошибку. Задача — сделать так, чтобы ответ отражал реальность.

ЧТО ДОЛЖНО ПОЛУЧИТЬСЯ

GET `/health` (без авторизации, как сейчас) проверяет две зависимости и отдаёт:

```
{
  "status": "ok",
  "checks": {
    "db": { "status": "ok", "latency_ms": 3 },
    "ml": { "status": "ok", "latency_ms": 41 }
  }
}
```

- Всё живо — код ответа **200**, `status: "ok"`.
- Хотя одна зависимость недоступна — код ответа **503**, `status: "down"`, у сломанной зависимости `"status": "down"` и короткое человеческое описание причины в поле `error`.
- База проверяется пингом пула соединений, ML-сервис — запросом на его собственный `/health` (он там есть).
- У всей проверки общий таймаут **2 секунды**: ручка здоровья не имеет права висеть, даже если зависимость висит.

ГРАНИЦЫ

Менять: `backend/internal/http/handlers/health_handler.go`, `backend/internal/http/router.go`, `backend/internal/app/app.go` (проброс зависимостей), `backend/cmd/api/main.go` (только если без этого не собрать), `backend/internal/client/ml_client.go` (только добавить метод проверки, существующие не переписывать). **Создать:** `backend/internal/http/handlers/health_handler_test.go`.

Нельзя трогать: `backend/internal/cian/`, `habitus/`, `frontend/`, все остальные обработчики и сервисы.

ПРИЁМКА — ПРОВЕРЯТЬ ПО ПУНКТАМ

1. `cd backend && go test ./...` — всё зелёное, тестов стало больше.
2. При поднятом проекте `curl -i localhost:8080/health` отдаёт **200** и обе зависимости в состоянии `ok`.
3. Остановить базу: `docker compose stop db`. Тот же запрос отдаёт **503**, у `db` состояние `down`, у `ml` по-прежнему `ok`.
4. Ответ приходит **быстро** (в пределах пары секунд), а не висит. Проверить: `time curl -s localhost:8080/health`.
5. Поднять базу обратно: `docker compose start db`. Через несколько секунд снова 200.
6. Остальной API не сломался: `curl -s -b /tmp/habitus.cookie localhost:8080/api/v1/me` отвечает как раньше.
7. `git show --stat HEAD` — изменены только файлы из раздела «Границы».

Проект Habitus, Go-шлюз в папке backend/. Fiber, pgx (pgxpool), тесты стандартным go test.

Задача: сделать ручку GET /health честной. Сейчас она в backend/internal/http/handlers/health_handler.go состоит из одной строки и всегда возвращает {"status":"ok"} — даже когда база недоступна.

Требования:

1. Проверять две зависимости: базу — пингом пула pgxpool; ML-сервис — HTTP-запросом на его собственный GET /health (он там есть, адрес сервиса уже лежит в конфиге, см. internal/config).
2. Формат ответа:
{"status":"ok","checks":{"db":{"status":"ok","latency_ms":3},"ml":{"status":"ok","latency_ms":41}}}
3. Все зависимости живы — HTTP 200 и status "ok". Хотя бы одна недоступна — HTTP 503, status "down", а у сломанной зависимости status "down" и поле error с коротким понятным описанием причины.
4. Общий таймаут всей проверки — 2 секунды. Ручка здоровья не должна висеть, даже если зависимость висит. Используй context.WithTimeout.
5. Обе проверки выполняй параллельно, а не одну за другой.
6. Ручка остаётся без авторизации и по тому же адресу /health.
7. Напиши тесты на подставных зависимостях (без реальной базы и без реальной сети): всё хорошо → 200; база лежит → 503 и db.status == "down"; ML лежит → 503 и ml.status == "down"; зависимость висит дольше таймаута → ручка всё равно отвечает, а не зависает.

Границы:

- меняй: backend/internal/http/handlers/health_handler.go, backend/internal/http/router.go, backend/internal/app/app.go, backend/cmd/api/main.go (только если иначе не собирается), backend/internal/client/ml_client.go (только добавь метод проверки, существующие методы не переписывай)
- создай: backend/internal/http/handlers/health_handler_test.go
- не трогай backend/internal/cian/, habitus/, frontend/, остальные обработчики и сервисы
- зависимости передавай в обработчик через интерфейсы, чтобы их можно было подменить в тестах

Правила проекта:

- слои не смешивать: SQL живёт в internal/repository, логика в internal/service, разбор запроса и ответ в internal/http/handlers
- перед завершением: cd backend && go test ./... — всё зелёное
- комментарии по-русски и только там, где объясняют «почему», а не «что»
- один коммит, сообщение по-русски в формате Conventional Commits, например «feat: /health проверяет базу и ML-сервис»
- в коммите не должно быть Co-Authored-By и любых других трейлеров

Сначала покажи план: какие файлы тронешь, как пробросишь зависимости и какие тесты напишешь. Код начинай писать только после того, как я подтверждаю план.

ЕСЛИ ПОШЛО НЕ ТАК

- Тесты требуют настоящую базу → «Тесты не должны ходить в реальную базу и сеть. Опиши зависимости интерфейсами и подставь заглушки.»
- Ручка висит при остановленной базе → «Таймаут не работает. Проверь, что контекст с таймаутом реально доходит до проверки, и добавь тест на зависшую зависимость.»
- При лежащей базе отвечает 200 → «Если хоть одна зависимость down, код ответа обязан быть 503. Добавь тест.»

Задача 2 — Фильтры для объявлений на карте

ОБЛАСТЬ	ОЦЕНКА	ПОРЯДОК
Go, backend/	1–2 дня	После задачи 1

Начинать после того, как задача 1 влита в `main`, и сделав `git pull`.

ЗАЧЕМ

Ручка `GET /api/v1/geo/listings` отдаёт карте все объявления под видимой областью — без разбора. Треть из них (667 из 2143) вообще без фотографий, и на карте они выглядят пустыми карточками. Фильтровать это на стороне сайта неправильно: лимит в 2000 объектов срабатывает **до** фильтрации, и нужные объекты просто не доезжают. Фильтры должны применяться в запросе к базе.

ЧТО ДОЛЖНО ПОЛУЧИТЬСЯ

Ручка принимает дополнительные необязательные параметры:

ПАРАМЕТР	ЗНАЧЕНИЕ	СМЫСЛ
<code>has_photo</code>	<code>true</code>	только объявления, у которых есть хотя бы одна фотография
<code>price_min</code> , <code>price_max</code>	число	границы цены в рублях, включительно
<code>rooms</code>	<code>1,2,3</code>	число комнат, список через запятую
<code>limit</code>	число	сколько отдавать, по умолчанию 2000, больше 2000 нельзя

- Фильтры складываются: указано несколько — выполняются все сразу.
- В ответ добавляется поле `count` — сколько объектов реально вернулось.
- Параметры не указаны — поведение **ровно как сейчас**. Это важно: сайт уже пользуется этой ручкой, и он не должен ничего заметить.
- Кривое значение параметра (`price_max=abc`) — параметр игнорируется, ошибки нет. Так уже сделано с `bbox`: в этой ручке кривой ввод не роняет карту, а просто не учитывается. Логiku придумывать не надо, надо повторить существующую.

ГРАНИЦЫ

Менять: `backend/internal/repository/listing_repo.go` (метод `ListInBBox`), `backend/internal/service/geo_layers_service.go`, `backend/internal/http/handlers/geo_handler.go`, `backend/internal/service/geo_layers_service_test.go`. **Создать:** тест на разбор параметров рядом с обработчиком.

Нельзя трогать: `backend/internal/cian/`, `habitus/`, `frontend/`, метод `Layers` и всё, что касается слоёв карты (шум, школы, бары) — работаем только со списком объявлений.

ПРИЁМКА — ПРОВЕРЯТЬ ПО ПУНКТАМ

- `cd backend && go test ./...` — зелёное, тестов стало больше.
- Без параметров ответ такой же, как до задачи (сравнить количество объектов до и после).
- `...&has_photo=true` — объектов стало заметно меньше, и **у каждого** в ответе непустая обложка. Проверить глазами несколько штук.
- `...&price_max=15000000` — в ответе нет ни одной цены выше 15 млн.
- `...&rooms=1,2` — в ответе только однушки и двушки.
- Комбинация `...&has_photo=true&rooms=2&price_max=20000000` — выполняются все три условия одновременно.
- `...&limit=5` — ровно 5 объектов, и `count` равен 5.
- `...&limit=999999` — не больше 2000.
- `...&price_max=abc` — ответ 200, параметр просто проигнорирован, ошибки нет.
- Сайт продолжает работать: открыть `localhost:3000`, сделать поиск — точки на карте на месте.

11. `git show --stat HEAD` — только файлы из «Границ».

ПРОМТ ДЛЯ CLAUDE CODE — КОПИРОВАТЬ ЦЕЛИКОМ

Проект Habitus, Go-шлюз в папке backend/. Fiber, pgx, PostgreSQL с PostGIS, тесты стандартным go test.

Задача: добавить фильтры в ручку GET /api/v1/geo/listings — она отдаёт карте объявления под видимой областью. Сейчас фильтров нет вовсе.

Требования:

1. Новые необязательные параметры запроса:
has_photo=true — только объявления, у которых есть хотя бы одна фотография (колонок photos, массив: не NULL и длина больше нуля)
price_min, price_max — границы цены в рублях, включительно
rooms=1,2,3 — число комнат, список через запятую
limit — сколько отдавать; по умолчанию 2000, значение больше 2000 обрезается до 2000
2. Все фильтры должны применяться в SQL-ЗАПРОСЕ, а не в Go после выборки. Это принципиально: лимит срабатывает до фильтрации, и при фильтрации в Go нужные объекты просто не доедут.
3. В JSON-ответ добавь поле count — количество вернувшихся объектов.
4. Если ни один параметр не указан, поведение должно остаться ровно таким, как сейчас: этой ручкой уже пользуется сайт, и он не должен заметить изменений.
5. Кривое значение параметра (например price_max=abc) — параметр молча игнорируется, ошибки нет. Именно так уже сделано с параметром bbox в этом же обработчике: посмотри функцию parseBbox и повтори её подход, не выдумывая своего.
6. Параметры фильтра передавай от обработчика к сервису и репозиторию одной структурой, а не пятью аргументами.
7. Тесты: на разбор параметров запроса (включая кривые значения и обрезание limit до 2000) и на то, что сервис прокидывает фильтры в репозиторий. Тесты на подставном репозитории, реальная база не нужна — посмотри, как это уже сделано в backend/internal/service/geo_layers_service_test.go, и делай так же.

Границы:

- меняй: backend/internal/repository/listing_repo.go (метод ListInBBox), backend/internal/service/geo_layers_service.go, backend/internal/http/handlers/geo_handler.go, backend/internal/service/geo_layers_service_test.go
- создай при необходимости тест рядом с обработчиком
- не трогай метод Layers и всё, что связано со слоями карты (шум, школы, бары) — работаем только со списком объявлений
- не трогай backend/internal/cian/, habitus/, frontend/

Правила проекта:

- таблица listings принадлежит Python-части, Go в неё ТОЛЬКО читает: никаких INSERT, UPDATE, DELETE
- схему базы не менять, все нужные колонки уже есть
- SQL живёт только в internal/repository, логика в internal/service, разбор запроса в internal/http/handlers
- координаты везде [lng, lat], EPSG:4326 — порядок не переставлять
- параметры в SQL передавать только через плейсхолдеры pgx (\$1, \$2 ...), никакой склейки строк запроса
- перед завершением: cd backend && go test ./... — всё зелёное
- комментарии по-русски и только там, где объясняют «почему»
- один коммит, сообщение по-русски в формате Conventional Commits, например «feat: фильтры по фото, цене и комнатам для объявлений на карте»
- в коммите не должно быть Co-Authored-By и любых других трейлеров

Сначала прочитай текущий код ручки и репозитория и покажи план: какие файлы тронешь, как будет выглядеть SQL и какие тесты напишешь. Код — после моего подтверждения.

ЕСЛИ ПОШЛО НЕ ТАК

- Фильтрует в Go после выборки → «Фильтрация должна быть в SQL. В Go после выборки нельзя: лимит срабатывает раньше и объекты теряются.»
- Собирает SQL склейкой строк → «Не склеивай SQL из строк. Только плейсхолдеры \$1, \$2 и аргументы pgx.»
- Кривой параметр приводит к ошибке 400 → «Кривое значение параметра должно игнорироваться, как это сделано с bbox в этом же файле. Ответ остаётся 200.»

- Без параметров ответ изменился → «Поведение без параметров обязано остаться прежним. Добавь тест, который это стережёт.»

Задача 3 — Эндпоинт статистики данных

ОБЛАСТЬ

Go, backend/

ОЦЕНКА

1–2 дня

ПОРЯДОК

После задачи 2

Начинать после того, как задача 2 влита в `main`, и сделав `git pull`.

ЗАЧЕМ

Сбор объявлений идёт по расписанию каждые 6 часов. Чтобы понять, в каком состоянии данные после очередного сбора — сколько объявлений, у скольких есть фотографии и адреса, когда было последнее обновление, — сейчас приходится руками писать запросы к базе. Нужна одна ручка, которая отдаёт это целиком. Это первая задача, где вы делаете эндпоинт от базы до ответа, поэтому она идёт последней.

ЧТО ДОЛЖНО ПОЛУЧИТЬСЯ

GET `/api/v1/stats` (под авторизацией, как остальные ручки в `/api/v1`) отдаёт:

```
{
  "listings": {
    "total": 2143,
    "with_address": 2143,
    "with_photos": 1476,
    "with_metro": 1937,
    "with_coordinates": 2143,
    "by_source": { "cian": 2143 },
    "last_updated": "2026-08-17T11:38:04Z"
  },
  "photos": { "total": 31766, "avg_per_listing_with_photos": 21.5 },
  "layers": { "noise": 46335, "communal": 4501, "crime": 4466 },
  "points": { "alcohol": 3048, "school": 2351, "park": 1504, "bar": 1432, "metro": 276 }
}
```

- Считаются только активные объявления (`is_active`).
- Городские слои берутся из таблицы `urban_evidence` с разбивкой по колонке `layer`, точки на карте — из таблицы `poi` с разбивкой по `kind`.
- На пустой базе отдаются нули, а не ошибка и не деление на ноль.
- Если объявлений с фотографиями нет, поле среднего **не отдаётся вовсе** — ноль здесь означал бы «в среднем ноль фотографий», а это неправда.
- Пересчёт целиком в базе, несколькими запросами — не тащить строки в Go и не считать их там.

ГРАНИЦЫ

Создать: `backend/internal/repository/stats_repo.go`, `backend/internal/service/stats_service.go`, `backend/internal/http/handlers/stats_handler.go` и тесты рядом с сервисом и обработчиком. **Менять:** `backend/internal/http/router.go` (одна строка маршрута), `backend/internal/app/app.go` (проброс зависимости).

Нельзя трогать: `backend/internal/cian/`, `habitus/`, `frontend/`, существующие репозитории, сервисы и обработчики.

ПРИЁМКА — ПРОВЕРЯТЬ ПО ПУНКТАМ

1. `cd backend && go test ./...` — зелёное, тестов стало больше.
2. `curl -s -b /tmp/habitus.cookie localhost:8080/api/v1/stats | python3 -m json.tool` — печатается корректный JSON нужной формы.
3. Числа сходятся с реальностью: доля объявлений с фотографиями около 69%, со станцией метро около 90%.
4. Без куки авторизации ручка отвечает ошибкой доступа, а не данными.
5. Ручка отвечает быстро — меньше секунды. Проверить: `time curl -s -b /tmp/habitus.cookie localhost:8080/api/v1/stats`.

6. Остальной API не сломался: поиск на сайте по-прежнему работает.
7. Есть тест на пустую базу: нули, отсутствие среднего числа фотографий и никакой паники.
8. `git show --stat HEAD` — только файлы из «Границ».

ПРОМТ ДЛЯ CLAUDE CODE — КОПИРОВАТЬ ЦЕЛИКОМ

Проект Habitus, Go-шлюз в папке backend/. Fiber, pgx, PostgreSQL, тесты стандартным go test.

Задача: добавить ручку GET /api/v1/stats — отчёт о полноте данных в базе.

Требования:

1. Формат ответа:

```
{ "listings": { "total": 2143, "with_address": 2143, "with_photos": 1476, "with_metro": 1937, "with_coordinates": 2143, "by_source": { "cian": 2143 }, "last_updated": "2026-08-17T11:38:04Z", "photos": { "total": 31766, "avg_per_listing_with_photos": 21.5 }, "layers": { "noise": 46335, "communal": 4501, "crime": 4466 }, "points": { "alcohol": 3048, "school": 2351, "park": 1504, "bar": 1432, "metro": 276 } }
```
2. Источники данных: таблица listings (считать только строки с is_active), таблица urban_evidence с разбивкой по колонке layer, таблица poi с разбивкой по колонке kind. Сначала посмотри, какие колонки в этих таблицах реально есть, и работай только с ними.
3. last_updated — максимум колонки updated_at по активным объявлениям.
4. На пустой базе возвращаются нули: ни ошибки, ни паники, ни деления на ноль.
5. ВАЖНО: если объявлений с фотографиями нет, поле avg_per_listing_with_photos не отдаётся вообще (пропусти его в JSON). Ноль здесь означал бы «в среднем ноль фотографий на объявление», то есть выдуманный факт, а в этом проекте так делать запрещено.
6. Всё считается запросами к базе — агрегатами SQL. Не выгружать строки в Go и не считать их циклом.
7. Ручка живёт под авторизацией, в той же группе /api/v1, что и остальные.
8. Тесты на подставном репозитории, без реальной базы: обычный случай; пустая база (нули, без поля среднего, без паники); ошибка репозитория корректно превращается в ошибку ответа. Как писать такие тесты — смотри backend/internal/service/geo_layers_service_test.go.

Границы:

- создай: backend/internal/repository/stats_repo.go, backend/internal/service/stats_service.go, backend/internal/http/handlers/stats_handler.go и тесты рядом с сервисом и обработчиком
- меняй: backend/internal/http/router.go (добавь маршрут), backend/internal/app/app.go (пробрось зависимость)
- не трогай существующие репозитории, сервисы и обработчики
- не трогай backend/internal/cian/, habitus/, frontend/

Правила проекта:

- таблицы listings, urban_evidence и poi принадлежат Python-части, Go в них ТОЛЬКО читает: никаких INSERT, UPDATE, DELETE
- схему базы не менять, все нужные колонки уже есть
- слои не смешивать: SQL только в internal/repository, логика в internal/service, разбор запроса и ответ в internal/http/handlers
- параметры в SQL только через плейсхолдеры pgx, без склейки строк
- перед завершением: `cd backend && go test ./... — всё зелёное`
- комментарии по-русски и только там, где объясняют «почему»
- один коммит, сообщение по-русски в формате Conventional Commits, например «feat: ручка статистики полноты данных»
- в коммите не должно быть Co-Authored-By и любых других трейлеров

Сначала посмотри реальные колонки таблиц и покажи план: какие файлы создашь, какие запросы напишешь и какие тесты. Код — после моего подтверждения.

ЕСЛИ ПОШЛО НЕ ТАК

- Выгружает все строки и считает в Go → «Считай агрегатами в SQL. Выгружать таблицу в Go нельзя — там десятки тысяч строк.»
- На пустой базе паника или ошибка → «На пустой базе должны быть нули. Почини деление на ноль и добавь тест.»
- Отдаёт среднее число фотографий равным нулю → «Если объявлений с фото нет, поле не отдаётся вовсе. Ноль — выдуманный факт, в проекте это запрещено. Добавь тест.»

- Предлагает добавить колонку или индекс → «Схему базы менять нельзя. Считай по тем колонкам, которые уже есть.»

Как принимать работу

Три минуты на задачу. Делается после того, как разработчик сказал «готово».

```
git pull
git log --oneline -3          # один коммит, сообщение по-русски, без подписей
git show --stat HEAD         # изменены только файлы из раздела «Границы»
cd backend && go test ./...   # всё зелёное
```

Затем пройти пункты приёмки из раздела задачи руками — все, не выборочно. Каждый пункт проверяется за несколько секунд.

ОТДЕЛЬНО ПРОВЕРИТЬ, ЧТО НЕ СЛОМАЛОСЬ СОСЕДНЕЕ

```
curl -s -b /tmp/habitus.cookie \
  'localhost:8080/api/v1/geo/listings?bbox=37.5,55.70,37.70,55.80' \
  | python3 -m json.tool | head -20
# и открыть localhost:3000, сделать любой поиск – выдача и точки на карте на месте
```

Красные флаги в ответах Claude Code

Любой из этих признаков — повод не принимать работу и переспросить:

ПРИЗНАК	ЧТО ЭТО ЗНАЧИТ
Удалил или закомментировал тест, чтобы «прошло»	Скрыл поломку вместо починки. Откатывать.
В коде появился <code>INSERT</code> , <code>UPDATE</code> или <code>DELETE</code> по таблице <code>listings</code>	Go в эту таблицу только читает. Прямое нарушение.
SQL собирается склейкой строк вместо плейсхолдеров	Дыра в безопасности. Переделать.
Тронул файлы вне раздела «Границы»	Расползание задачи. Откатить лишнее.
Изменился <code>habitus/db/schema.sql</code>	Схему базы в этих задачах не трогаем вовсе.
Подставил <code>0</code> вместо отсутствующего значения	Нарушение правила проекта: поле должно отсутствовать.
Тесты стали требовать живую базу или сеть	Раньше не требовали. Значит, зависимости не подменяются.
В коммите строка <code>Co-Authored-By</code>	Правило проекта. Переделать коммит.
Изменился <code>.env</code>	Риск утечки ключей. Остановиться и сообщить Ярику.
«Возможно», «должно работать», «попробуйте проверить»	Он не проверял. Требовать запустить тесты и показать вывод.

Когда останавливаться и писать

- Две попытки починить одно и то же не помогли.
- Требуется изменить схему базы, Python-часть или сборщик объявлений — значит, задача понята неправильно.
- Сломалось что-то за пределами своей задачи: перестали работать поиск, вход, карта.
- В `git status` появились файлы, которых вы не касались.

Писать с полным текстом ошибки и последним отправленным промтом. Не пересказом.